

第3章 数组与链表

1.结构体

结构体是自定义数据类型，它是必须存在的，还是仅仅只为方便？前面我说过，要从设计的角度去理解一门语言。

我的答案是，结构体是必须存在的。

```
1  typedef struct{
2      int id;
3      char name[30];
4      int age;
5      int sex;
6  }Person1;
7
8
9
10 typedef struct{
11     int id;
12     char *name;
13     int age;
14     int sex;
15 }Person2;
16
17
18 // 以上定义了两个结构体，它们的赋值难点主要集中在 name[30] 和 *name 上。
19
20 int main(){
21     Person1 wzd1 = {0}; // 初始化为0
22     Person2 wzd2 = {0};
23
24     // 现在要给这两个结构体中的name赋值。
25
26     //-----wzd1-----
27     wzd1.name = "wangzhendong"; //错误。
28
29     // 正确的是↓
30     int i = 0;
31     char *name = "wangzhendong";
32     for(i;i<strlen(name);i++){
33         wzd1.name[i] = name[i];
34     }
35
36
```

```

37
38     //-----wzd2-----
39
40     wzd2.name = "wangzhendong";//正确。
41     // 实际上，wzd2.name指针指向的是main函数的局部空间中的存储
    了"wangzhendong"的首地址。
42     // 你还可以在堆上分配内存↓
43     char *name = (char*) malloc(30);
44     memset(name, '\0', 30);
45     wzd2.name = name;
46
47     return 0;
48 }
49

```

上面写了这么多代码，其实就是想说，一个结构体定义好以后，它在内存中的构造是什么样的？是在一个区域，还是在多个区域？

2.结构体中next指针

我上课时，**搞错了一点**，带next指针的结构体的写法应该如下：

```

1  typedef struct{
2      int id;
3      char *name;
4      int age;
5      int sex;
6      struct Person* next; //这个地方的写法是这样的。
7  }Person;

```

我们需要回答下面的问题：

- 为什么 上面的第6行必须要指针，而不能向下面这样定义

```

1  typedef struct{
2      int id;
3      char *name;
4      int age;
5      int sex;
6      struct Person next; //为什么这个地方必须要指针
7  }Person;

```

3.数值结构体

结构体是自定义数据类型，它本质上和int、float等并无两样。我们完全可以定义一个

```
1  typedef struct{
2      int id;
3      char *name;
4      int age;
5      int sex;
6  }Person;
7
8
9  int main(){
10     Person all[10]; //定义一个结构数组
11     return 0;
12 }
```

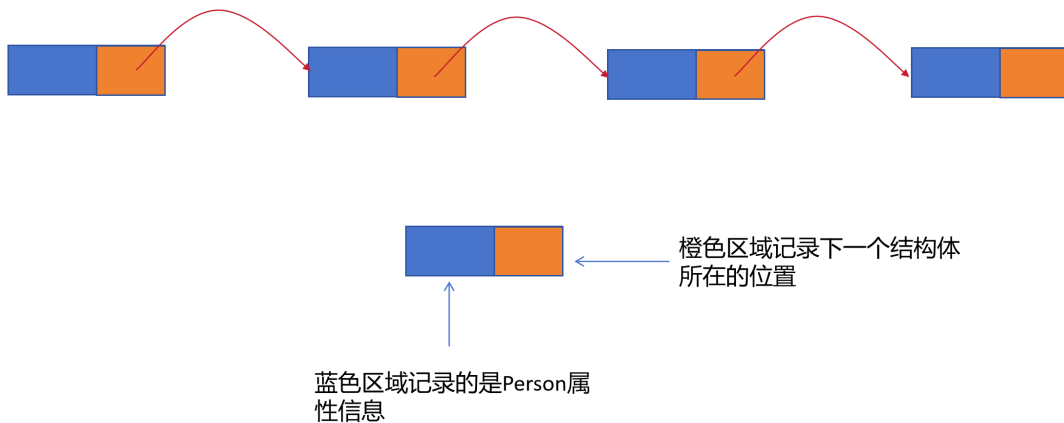
4.构建链表

我们将在后面学习很多的数据结构，如树、图、栈、队列等等，它们均是逻辑上的。逻辑对应物理，物理上的数据结构只有两种——数组和链表。

```
1  typedef struct{
2      int id;
3      struct Person *next;
4  }Person;
5
6
```

```
1  //构建链表
2  int main(){
3      //定义节点并赋值初始化。
4      Person head = {100,0}, node1 = {101,0}, node2 = {102,0}, tail
= {999,0};
5      head.next = &node1;
6      node1.next = &node2;
7      node2.next = &tail;
8  }
```

有了next指针，以上四个结构体就有了一层像链条一样的关系，如下图：



一个链表，关键在于可遍历。

```

1 // 遍历
2 Person *p = &head; // 定义一个指针变量，相当于 for中(int
  i;i<N;i++)
3 while(p != 0){
4     printf("%x ",p);
5     p = p->next;
6 }

```

光遍历是不够的，还要支持插入。

```

1 Person node3 = {103,0};
2 // 插入
3 p = &head; //从头开始找
4 Person *temp = &head; //临时变量 用来存储上一个p，因为链表不能返回
  嘛
5 while( p != 0){
6     if(p->id > node3.id){
7         temp->next = &node3;
8         node3.next = p;
9         break;
10    }
11    temp = p; //temp存储p
12    p = p->next; //p变成下一个p
13 }
14
15 //还有一种情况，如果这个node3要插入最末尾，这个特征是，p == 0
16
17 if(p == 0){
18     temp->next = &node3;
19 }
20

```

下面是修改

```
1 //修改是最简单的，因为这个链不发生变化。我就不写了
```

下面是删除

```
1 //删除 假设我们要删除id=102的节点
2 p = &head; //从头开始找
3 temp = &head;
4 while(p != 0){
5     if(p->id == 102){ // 删除的前提条件是信息匹配
6         temp->next = p->next;
7         break;
8     }
9     temp = p;
10    p = p->next;
11 }
```

增加、删除有一个问题，假设要在增加的是头节点，或者删除的是头节点怎么办？上面的代码中，总是认为head为头节点，但在前面增加一个节点或者删除它后，链表的头就再也找不到了。

因此，要定义一个指针变量，专门用来存储头节点。

```
1 int main(){
2     Person *head = 0; // 这个指针始终保留
3
4     Person node0 = {0}, node1 = {0}, node2 = {0}, tail = {0};
5
6     head = node0; // 让这个头指针永远指向链表的头部。
7
8     return 0;
9 }
```

5.利用函数实现链表的CURD

```
1 #include <stdio.h>
2 typedef struct Person{
3     int id;
4     struct Person* next;
5 }Person;
6
7
8 /*追加，相当于create并插入到末尾*/
9 Person *append(int id, Person *head){
10     Person *node = (Person*) malloc(sizeof(Person)); //新建一个节点
11     memset(node,0,sizeof(Person));
12     node->id = id;
```

```

13
14     if(head == 0){ // 如果是空链表，
15         head = node;
16         return head;
17     }
18
19     Person *p = head; // 如果不是空链表，要插入到末尾
20     while(p->next != 0){ // 这面用了 p->next，如果看不懂就老老实实的用temp指针
21         p = p->next;
22     }
23     p->next = node;
24     return head;
25 }
26 /**
27  * 有兴趣的同学可以思考一下，为什么append函数要返回head指针。
28  */
29
30
31
32
33
34 Person * research(int id, Person *head){
35     if(head == 0)
36         return 0;
37     Person *p = head;
38     while(p != 0){
39         if(p->id == id){
40             return p;
41         }
42         p = p->next;
43     }
44     return 0;
45 }
46
47
48 /**修改内容*/
49 void update(int id, Person* head, int newId){
50     if(head == 0) return;
51     Person *p = research(id,head);
52     if(p == 0) return;
53     p->id = newId;
54 }
55
56
57
58 /**删除*/
59 Person* del(int id, Person *head){
60     if(head == 0)

```

```

61     return head;
62     Person *p = head;
63     if(head->id == id){ // 删除头节点, head要重新指向第二节点
64         head = head->next;
65         free(p);
66         return head;
67     }
68     Person *temp = head;
69     while(p != 0){
70         if(p->id == id){
71             temp->next = p->next;
72             free(p);
73             break;
74         }
75         temp = p;
76         p = p->next;
77     }
78
79     return head;
80 }
81
82
83
84
85
86
87 /**遍历 我们总是用这个函数来测试*/
88 void traverse(Person *head){
89     Person *p = head;
90     while(p!=0){
91         printf("%d, ",p->id);
92         p = p->next;
93     }
94 }
95
96 int main(){
97     Person *head = 0;
98
99     int message[10] = {0};
100    int i = 0;
101    for(;i<10;i++){
102        message[i] = 1000+i;
103    }
104
105    /**构建十个元素的数组*/
106    for(i=0;i<10;i++){
107        head = append(message[i],head);
108    }
109

```

```
110
111     Person *p = research(1001,head);
112     if(p == 0){
113         printf("没有找到, id = 1005 \n");
114     }else{
115         printf("%d \n",p->id);
116     }
117
118     update(1005,head,1015);
119
120
121     //head = del(1015,head);
122     head = del(1000,head);
123     traverse(head);
124
125     return 0;
126 }
```